

Blazor Treasure

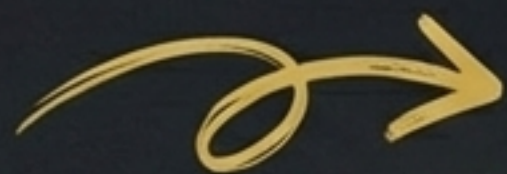
연금술사의 보물 지도: .NET 10 시대의 Blazor 활용법

모든 위대한 여정의 시작

- * 예체능 학과 출신
- * 비전공자
- * Unity로 C# 과 .NET 입문

관점의 전환: 약점을 강점으로

예체능 비전공자



감성과 논리를 잇는
하이브리드 개발자

Unity로 시작



C#의 시각적 즐거움을
아는 **개발자**

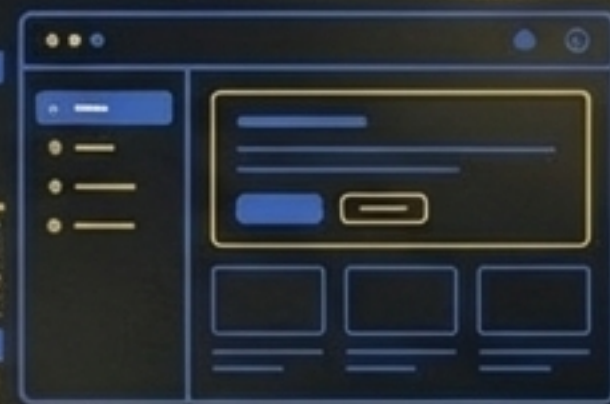
기술로 사회 발전에 관심



실질적인 사회 문제를 해결하는,
가치 있는 기술을 지향합니다

마법의 도구, Blazor

Blazor는 JavaScript 대신 **C#을 사용하여**
대화형 웹 UI를 구축할 수 있는 프레임워크입니다.



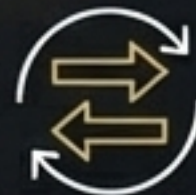
완벽한 ASP.NET
Core 통합



재사용 가능한
컴포넌트 모델



직관적인 UI
이벤트 처리



양방향 데이터
바인딩

.NET 10으로 더욱 강력해진 Blazor



상태 지속성 향상



렌더링 모드 일관성



다층 폼 유효성 검사



JavaScript 상호 운용성 향상



패스키(Passkey) 지원

연금술의 핵심: Blazor 렌더링 생명주기



1단계: 서버의 첫 숨결

사전 렌더링 (Pre-rendering)



서버에서 순수 HTML을 생성하는 과정

2단계: 클라이언트와의 연결

SignalR 렌더링 (SignalR Rendering)



3단계: 살아있는 상호작용



Code 렌더링 (타이머 등)



Event 렌더링 (버튼 클릭 등)

ShouldRender → ShouldRender

OnAfterRenderAsync (firstRender: false) → OnAfterRenderAsync (firstRender: false)

왜 PreRendering이 필요한가?

서버에서 HTML을 빠르게 생성하여 즉시 표시
결과 → SEO에 절대적으로 유리



값비싼 연산을 다루는 기술

Challenge

고비용 렌더링: PreRendering 시 실행된 작업을 SignalR 연결 후 반복해야 하는 문제



Solution

'**.NET 10의 새로운 마법**':
'[PersistentState] 어트리뷰트



가장 중요한 것

이 모든 기술적 지식 너머에,
제가 'Blazor Treasure'를 만든 진짜 이유가 있습니다.

AI가 할 수 없는 일

기술은 문제를 해결하는 도구입니다.

하지만 어떤 문제를 풀어야 하는지,
그 가치를 어떻게 나눌 것인지를 결정하는 것은
결국 **사람의 몫**입니다.

지식을 나누고, 함께 성장하며, 더 나은 가치를 만드는 것.
그것이 바로 **연금술의 완성**입니다.

